



Departamento de Computación y Sistemas Inteligentes

Ingeniería de Sistemas

Seminario de Ingeniería de Software

Laboratorio desafío: App web desplegada en la nube con servicios open source y administrados

Profesores: Gonzalo Llano R Ph.D /José Luis Jurado Ph. D

Santiago de Cali, septiembre del 2026

Laboratorio desafío: CloudTasks — App web desplegada en la nube con servicios open source y administrados

1) Introducción

En este laboratorio se desarrollará **CloudTasks**, una aplicación web para la gestión de tareas personales o de un equipo de trabajo. El propósito no es únicamente construir una aplicación funcional, sino comprender cómo una solución de software puede evolucionar progresivamente desde un entorno local hasta una aplicación disponible en Internet utilizando servicios open source administrados de computación en la nube.

El laboratorio se desarrollará mediante **tres etapas progresivas**: **Etapla 1:** Se debe diseñar y desarrollar la app web utilizando **HTML, CSS y JavaScript**, aplicando buenas prácticas básicas de desarrollo y controlando la evolución del código mediante **Git**, para posteriormente almacenarlo y gestionarlo en **GitHub**.

Etapla 2: la aplicación debe ser desplegada en la nube mediante el uso de los servicios *open source* **Vercel** para construir y desplegar el **frontend**, y **Supabase** para el **Backend**, y de **persistencia de datos** con PostgreSQL. De esta manera, los estudiantes podrán observar la transición de una aplicación local hacia una solución en la nube que utiliza servicios administrados o *serverless*.

Etapla 3: Se debe incorporar en el pipeline el servicio *open source* de **Cloudflare** al flujo de **acceso seguro de la aplicación**. Se debe configurar el dominio, DNS, HTTPS y el acceso hacia la aplicación desplegada en Vercel, comprendiendo el papel que desempeña cada servicio dentro del flujo completo de la solución.

El laboratorio busca que el estudiante comprenda que en diseñar y desplegar una aplicación web en la nube, no consiste simplemente en "subir una aplicación a Internet", sino en **integrar diferentes servicios especializados para construir una solución funcional, accesible, escalable y gestionable**.

2) Objetivo general

Diseñar, desarrollar y desplegar una aplicación web funcional utilizando HTML, CSS y JavaScript, integrando Git, GitHub, Supabase, Vercel y Cloudflare para comprender progresivamente el ciclo de desarrollo, control de versiones, persistencia de datos, despliegue y acceso seguro a una aplicación en la nube.

3) Resultado de aprendizaje

Al finalizar el laboratorio, el estudiante estará en capacidad de: Diseñar e implementar una aplicación web básica e integrarla con servicios cloud administrados, aplicando control de versiones, persistencia de datos, despliegue y configuración de acceso mediante DNS y HTTPS, explicando la función y relación de cada servicio dentro de la arquitectura de la solución.

De manera específica, el estudiante deberá demostrar que puede:

- Desarrollar una aplicación web sencilla utilizando **HTML, CSS y JavaScript**.
- Aplicar **Git** para gestionar versiones del código fuente.
- Utilizar **GitHub** como repositorio remoto y plataforma de colaboración.
- Implementar persistencia de información utilizando **Supabase y PostgreSQL**.
- Desplegar una aplicación web utilizando **Vercel**.

- Configurar **Cloudflare** para gestionar DNS, HTTPS y el acceso seguro a la aplicación.
- Explicar el flujo de comunicación entre el usuario, Cloudflare, Vercel y Supabase.
- Identificar las responsabilidades de cada servicio dentro de la arquitectura.
- Documentar las decisiones técnicas y las evidencias del proceso.

4) Preparación del entorno y creación de cuentas

Antes de iniciar el desarrollo de la app web, se debe preparar el entorno de trabajo y disponer de las cuentas necesarias para utilizar las herramientas y servicios requeridos durante el laboratorio.

Todas las cuentas y servicios deberán utilizar inicialmente sus versiones, planes o niveles gratuitos (Free Tier/Free Plan), siempre que estén disponibles.

4.1 Instalar Git: Descargar e instalar **Git** en su computador local. Git será utilizado para:

- Crear el repositorio local del proyecto.
- Registrar cambios mediante commits.
- Crear y administrar ramas.
- Consultar el historial de modificaciones.
- Sincronizar el proyecto con GitHub.

4.2 Crear una cuenta en GitHub: Configurar una cuenta personal en **GitHub**, será utilizado como:

- Repositorio remoto y sistema de almacenamiento del código fuente.
- Plataforma de colaboración.
- Mecanismo de integración con Vercel.
- Evidencia del proceso de desarrollo mediante el historial del repositorio.
- Construir un repositorio denominado: **cloudtasks-equipoxX**

4.3 Crear una cuenta en Vercel: será utilizado para:

- Desplegar la aplicación web CloudTasks.
- Publicar la aplicación en Internet.
- Integrar el repositorio de GitHub.
- Automatizar nuevos despliegues cuando se actualice el código fuente.

4.4 Crear una cuenta en Supabase: será utilizado para:

- Crear el proyecto cloud.
- Utilizar una base de datos PostgreSQL administrada.
- Crear la tabla de tareas.
- Almacenar permanentemente la información de CloudTasks.
- Permitir que la aplicación JavaScript consulte y modifique los datos.

La aplicación deberá utilizar Supabase como servicio de un **Backend as a Service (BaaS)** administrado, evitando instalar y administrar localmente un servidor PostgreSQL para este laboratorio.

4.5 Crear una cuenta en Cloudflare: será utilizado principalmente para:

- Configurar DNS y CDN.
- Asociar un dominio o subdominio con la aplicación.
- Gestionar el acceso mediante HTTPS.
- Comprender el funcionamiento de un proxy inverso.
- Analizar el flujo de acceso entre el usuario, Cloudflare y Vercel.

5) Resumen de herramientas

Antes de iniciar las etapas del desafío, el grupo de trabajo deberá disponer de las siguientes herramientas necesarias para el desarrollo del proyecto:

Herramienta / servicio	Requerimiento	Uso en el laboratorio
Git	Descargar e instalar	Control de versiones
GitHub	Crear cuenta gratuita	Repositorio remoto
Vercel	Crear cuenta gratuita	Despliegue de la aplicación
Supabase	Crear cuenta gratuita	Base de datos y servicios backend
Cloudflare	Crear cuenta gratuita	DNS, HTTPS y acceso
Navegador web	Instalado	Desarrollo y pruebas
Editor de código	Instalado	Desarrollo HTML, CSS y JavaScript

Se recomienda utilizar un editor de código como **Visual Studio Code**, o el editor de su preferencia.

6) Recomendaciones importantes sobre las cuentas

6.1 Uso de cuentas personales

Cada estudiante deberá utilizar una cuenta propia para GitHub y podrá trabajar individualmente o como integrante de un equipo.

Cuando una plataforma permita trabajo colaborativo, se recomienda que el proyecto sea administrado por una cuenta o equipo claramente identificado y que todos los integrantes tengan acceso al repositorio correspondiente.

6.2 No utilizar información sensible: Se recomienda no almacenar en GitHub:

- Contraseñas y Tokens privados.
- Claves secretas y Credenciales de bases de datos.
- API Keys que deban mantenerse privadas.

6.3 Control de costos: El lab., está diseñado para ejecutarse usando los **planes gratuitos** de las plataformas.

- Verificar qué recursos están utilizando.
- Evitar habilitar servicios que impliquen costos innecesarios.
- No registrar métodos de pago si no son requeridos por la plataforma para el laboratorio.

- Eliminar recursos creados durante el laboratorio cuando ya no sean necesarios, si las condiciones de la plataforma lo requieren.

No se deberá asumir que un servicio es gratuito de manera indefinida: cada equipo es responsable de revisar las condiciones del plan gratuito vigente al momento de realizar el laboratorio.

7) **Descripción de la aplicación web CloudTasks:** una aplicación web sencilla para gestionar tareas, debe permitir:

- Crear una tarea.
- Visualizar las tareas registradas.
- Marcar una tarea como completada.
- Eliminar una tarea.
- Mostrar el estado de cada tarea.
- Validar los datos introducidos por el usuario.

Cada tarea deberá contener:

Campo	Descripción
id	Identificador único
title	Título de la tarea
description	Descripción
completed	Estado de la tarea
created_at	Fecha de creación
deadline	Fecha límite
priority	Prioridad

8) **Tecnologías y servicios:** La solución deberá utilizar:

Tecnología / servicio	Propósito
HTML	Estructura de la aplicación
CSS	Diseño y presentación
JavaScript	Lógica de la aplicación e interacción con servicios
Git	Control de versiones
GitHub	Repositorio remoto
Supabase	Backend administrado y persistencia
PostgreSQL	Base de datos de CloudTasks
Vercel	Despliegue y hosting de la aplicación
Cloudflare	DNS, HTTPS, CDN y acceso seguro

No se requiere utilizar frameworks como React, Angular o Vue. El propósito es que se comprenda los fundamentos de una aplicación web y su despliegue antes de introducir abstracciones adicionales.

9) Evolución del desarrollo y despliegue de la app

Una vez preparado el entorno, el desarrollo de la app web; CloudTasks se realizará en tres etapas:

ETAPA 1. Diseño, desarrollo y control de versiones

Propósito: Construir la primera versión funcional de CloudTasks en un entorno local y establecer un proceso básico de desarrollo y control de versiones.

Tecnologías: HTML, CSS, JavaScript, Git, GitHub



Actividades:

- Analizar los requisitos funcionales de CloudTasks.
- Diseñar la interfaz de usuario y crear la estructura HTML.
- Aplicar estilos utilizando CSS.
- Implementar la lógica utilizando JavaScript.
- Implementar las operaciones básicas de gestión de tareas.
- Probar la aplicación localmente.
- Crear un repositorio Git.
- Realizar commits durante el desarrollo.
- Crear el repositorio remoto en GitHub.
- Sincronizar el repositorio local con GitHub.

Estructura inicial sugerida

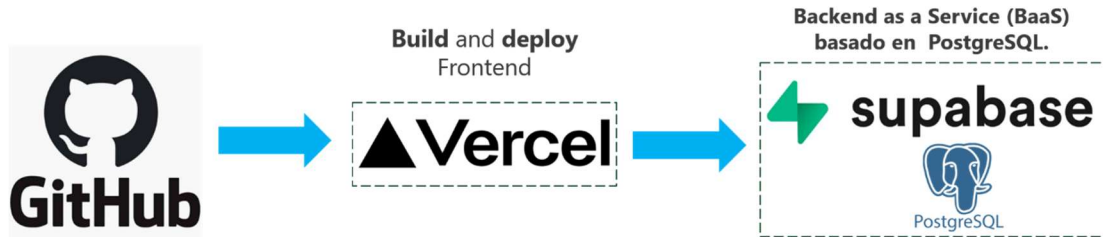
```
cloudtasks/  
├── index.html  
├── css/  
│   └── styles.css  
├── js/  
│   └── app.js  
├── README.md  
└── .gitignore
```

Resultado de la etapa: Al finalizar esta etapa se deberá disponer de la aplicación CloudTasks funcional ejecutándose localmente, y un repositorio GitHub que contenga el código fuente y el historial de desarrollo colaborativo.

ETAPA 2 — Desplegar la app en la nube

Propósito: Transformar la aplicación local en una aplicación web disponible en Internet e incorporar persistencia de información mediante un servicio cloud administrado.

Tecnologías: GitHub, Vercel, Supabase, PostgreSQL



La aplicación web será desplegada en **Vercel** y se incorporará **Supabase** como servicio de persistencia.

Actividad 2.1 — Base de datos en Supabase

- Crear el proyecto en Supabase.
- Crear la base de datos PostgreSQL.
- Diseñar la tabla tasks.
- Definir los atributos necesarios.
- Establecer el identificador de las tareas.
- Insertar registros de prueba.
- Consultar los registros.

Actividad 2.2 — Integración con JavaScript

La aplicación deberá dejar de utilizar únicamente información almacenada temporalmente en el navegador y comenzar a utilizar Supabase como sistema de persistencia. JavaScript deberá implementar operaciones CRUD.

Actividad 2.3 — Despliegue en Vercel

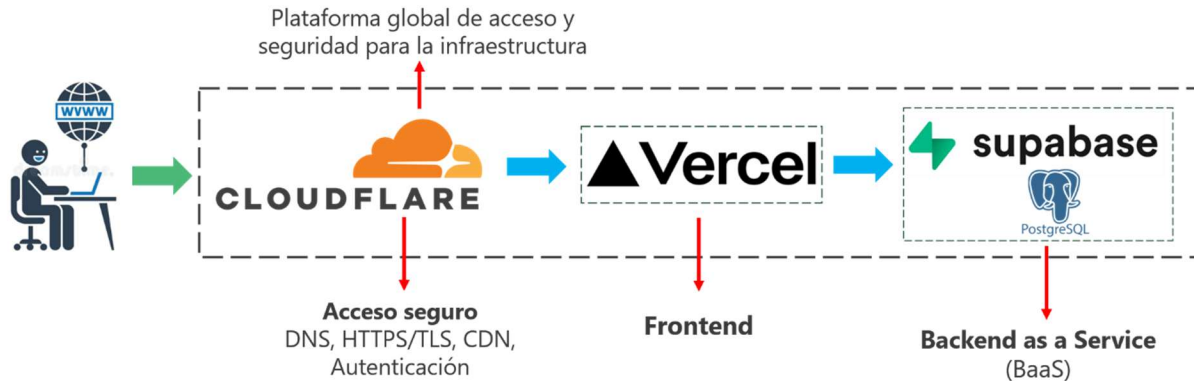
- Vincular el repositorio GitHub con Vercel.
- Configurar el proyecto.
- Realizar el primer despliegue.
- Obtener la URL pública.
- Probar CloudTasks desde Internet.
- Realizar una modificación en GitHub.
- Verificar el nuevo despliegue.

Resultado de la etapa

Una aplicación CloudTasks accesible públicamente mediante Vercel, conectada a una base de datos PostgreSQL administrada por Supabase y capaz de conservar las tareas registradas.

ETAPA 3. Integración del pipeline con Cloudflare, Vercel y Supabase

Propósito: Incorporar Cloudflare como componente de acceso seguro a la aplicación y comprender el flujo completo de una aplicación web desplegada en la nube.



Actividad 3.1 — Configuración de Cloudflare

El grupo deberá investigar y configurar los mecanismos necesarios para que Cloudflare permita el acceso seguro y eficiente a la aplicación, y además comprender conceptos como: **Dominio, DNS, Registro A y Registro CNAME, Nameservers, HTTPS, Certificado TLS, Reverse proxy, CDN.**

Actividad 3.2 — Integración Cloudflare → Vercel

Configurar el dominio utilizado por CloudTasks para que el acceso siga el flujo:

<https://cloudtasks.dominio.com> → Cloudflare → Vercel → CloudTasks

Actividad 3.3 — Validación del flujo completo

- El usuario puede acceder a la aplicación.
- Cloudflare resuelve el dominio.
- La aplicación está desplegada en Vercel.
- JavaScript se comunica con Supabase.
- Las tareas se almacenan en PostgreSQL.
- Los cambios realizados en GitHub pueden desencadenar nuevos despliegues en Vercel.

Resultado de la etapa

Al finalizar esta etapa se deberá disponer de: Una aplicación web CloudTasks accesible mediante un dominio configurado con Cloudflare, desplegada en Vercel y conectada a Supabase para la persistencia de información.

NOTA: Cada etapa deberá partir del resultado obtenido en la etapa anterior. De esta manera, el grupo podrá observar cómo una aplicación inicialmente local se transforma progresivamente en una **solución web desplegada e integrada mediante servicios cloud administrados.**

10) Entregables

Entregable 1 — Aplicación CloudTasks

Aplicación funcional desarrollada con: HTML, CS, JavaScript. Deberá implementar las operaciones básicas de gestión de tareas.

Entregable 2 — Repositorio GitHub: El repositorio deberá contener:

- Código fuente.
- README.md.
- .gitignore.
- Estructura organizada de archivos.
- Historial de commits.
- Ramas utilizadas durante el desarrollo.

Los commits deberán reflejar el proceso real de construcción de la aplicación.

Entregable 3 — Base de datos Supabase: Se deberá entregar evidencia de:

- Proyecto Supabase.
- Tabla tasks.
- Estructura de la tabla.
- Registros de prueba.
- Operaciones CRUD.
- Integración desde JavaScript.

Entregable 4 — Aplicación desplegada en Vercel: Se deberá proporcionar:

- URL pública de CloudTasks.
- Evidencia del despliegue.
- Evidencia de integración con GitHub.
- Evidencia de actualización automática después de un cambio en el repositorio.

Entregable 5 — Configuración Cloudflare: presentar evidencia de:

- Configuración DNS.
- Dominio o subdominio utilizado.
- Configuración HTTPS/TLS.
- Flujo Cloudflare → Vercel.
- Acceso exitoso a la aplicación.

Entregable 7 — Informe técnico

- a) Introducción, Objetivos.
- b) Requerimientos de CloudTasks.
- c) Diseño de la aplicación y Tecnologías utilizadas.
- d) Etapa 1: desarrollo local.

- e) Etapa 1: Git y GitHub.
- f) Etapa 2: Supabase.
- g) Etapa 2: Vercel.
- h) Etapa 3: Cloudflare.
- i) Arquitectura final.
- j) Flujo de datos.
- k) Flujo de despliegue.
- l) Pruebas realizadas.
- m) Uso de Inteligencia artificial ***
- n) Problemas encontrados y soluciones.
- o) Evidencias.
- p) Conclusiones.

Uso de inteligencia artificial ***

Se permite utilizar herramientas de IA durante el desarrollo del laboratorio, sin embargo, el uso de IA no reemplaza la responsabilidad del estudiante sobre el código desarrollado. El informe deberá incluir:

- Herramienta de IA utilizada.
- Propósito de utilización.
- Prompts utilizados.
- Código o solución obtenida.
- Cambios realizados por el grupo de estudiantes a las respuestas de la IA
- Proceso de validación.
- Errores encontrados.
- Cómo fueron solucionados.
- Reflexión sobre las limitaciones de la IA.

Durante la sustentación, el estudiante deberá ser capaz de **explicar el código y las decisiones técnicas tomadas**, independientemente de si recibió asistencia de una herramienta de IA.

11) Criterio general de evaluación

La evaluación tendrá en cuenta el funcionamiento de la app web, así como, el proceso de ingeniería utilizado para construirlo. Se valorará especialmente:

- Funcionalidad de CloudTasks.
- Calidad del código HTML/CSS/JavaScript.
- Uso adecuado de Git y GitHub.
- Persistencia e integración con Supabase.
- Despliegue correcto en Vercel.
- Configuración de Cloudflare.
- Comprensión de la arquitectura.
- Integración entre los diferentes servicios.
- Calidad de las pruebas.

- Documentación técnica.
- Capacidad para explicar las decisiones tomadas.
- Uso responsable y crítico de herramientas de IA.

El funcionamiento de la aplicación no será suficiente para obtener la máxima valoración: el estudiante deberá demostrar que comprende cómo se construyó, versionó, desplegó e integró la solución.